

恋语 (LianYu) 模块化开发文档

本文档帮助贡献者快速理解项目结构，高效并行开发。核心模块已用 ● 标注，修改时需格外谨慎。

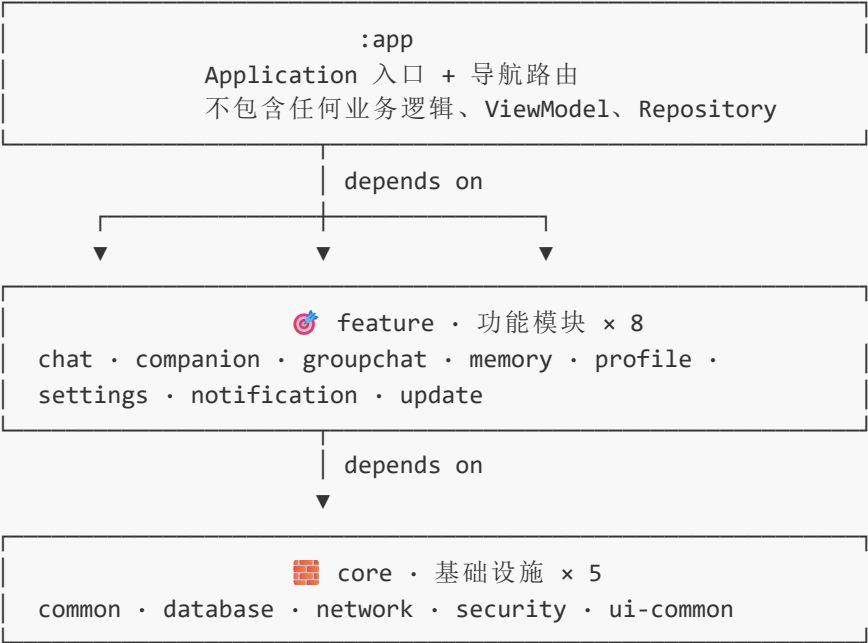
目录

- 架构概览
- 项目结构速览
- 依赖关系图
- 核心模块详解
 - :core:common — 公共基础
 - :core:database — 数据持久化
 - :core:network — 网络通信
 - :core:security — 安全加密
 - :core:ui-common — 通用 UI
- 功能模块详解
 - :feature:chat — 一对一聊天
 - :feature:companion — 伴侣管理
 - :feature:groupchat — 群聊
 - :feature:memory — 记忆系统
 - :feature:notification — 通知与后台
 - :feature:profile — 个人中心与首页
 - :feature:settings — 设置中心
- :app 入口模块
- 模块创建规范
- 跨模块通信规范
- 开发流程与最佳实践
- 常见问题
- 附录：完整文件清单

架构概览

恋语采用 **Feature-Based Modularization**（基于功能的模块化）架构，核心层（core）提供共享能力，功能层（feature）承载业务逻辑。

关键数字：14 个模块，1 个入口，5 个核心层，8 个功能层。



设计原则

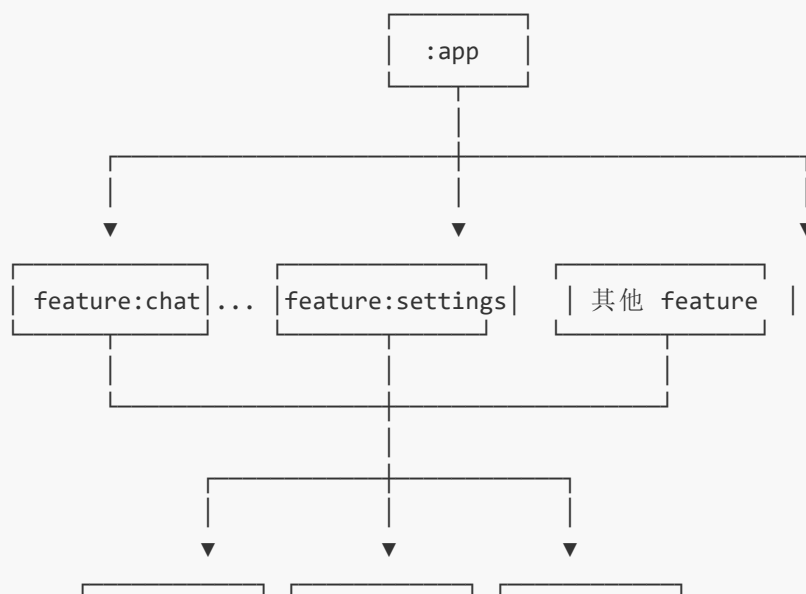
原则	说明
:app 轻量	仅保留 <code>Application</code> 、 <code>MainActivity</code> 和 Compose 导航路由，零业务逻辑
单向依赖	Feature → Core，禁止 Core → Feature，禁止 Feature 间直接依赖
高内聚低耦合	每个 Feature 包含完整的 Screen + ViewModel + Repository 链路
数据共享下沉	数据库 Entity/DAO/Repository 放在 <code>core:database</code> ，供多 Feature 共享
版本统一	所有依赖版本在 <code>gradle/libs.versions.toml</code> 集中管理

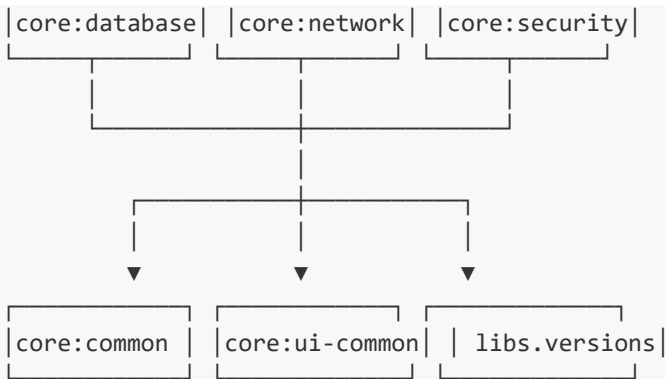
项目结构速览

```
LianYu/
├── app/                                # 📱 入口模块（Application + 导航）
│   ├── build.gradle.kts
│   └── src/main/
```

├─ AndroidManifest.xml	# ⚠️ Service/Receiver 用全限定类名
├─ java/com/lianyu/ai/	
│ └─ LianYuApplication.kt	# Application 初始化
│ └─ MainActivity.kt	# 唯一 Activity + NavHost
├─ res/	
│ └─ drawable/ic_notification.xml	
│ └─ xml/file_paths.xml	
│ └─ values/themes.xml	
├─ core/	# 🧩 核心层（5个模块）
│ └─ common/	# ● 公共基础工具
│ └─ database/	# ● Room 数据库 + 共享数据层
│ └─ network/	# ● AI 网络通信
│ └─ security/	# ● JNI 安全 + NDK 编译
│ └─ ui-common/	# ● 共享 UI 组件 + 主题
├─ feature/	# 🎯 功能层（8个模块）
│ └─ chat/	# 一对一聊天
│ └─ companion/	# AI 伴侣管理
│ └─ groupchat/	# 群聊
│ └─ memory/	# 记忆系统
│ └─ notification/	# 通知推送 + 后台服务
│ └─ profile/	# 首页 + 个人中心
│ └─ settings/	# 设置中心
│ └─ update/	# ⚠️ 未开源（含 API Token）
├─ gradle/	
│ └─ libs.versions.toml	# ● 版本号集中管理（所有模块共享）
├─ build.gradle.kts	# 根 build 配置
├─ settings.gradle.kts	# ● 模块注册（新增模块必须在这里 include）
└─ README.md	

依赖关系图





依赖规则表

规则	说明	检查方法
app → feature:*	app 依赖所有 feature 以注册 导航路由	查看 app/build.gradle.kts
app → core:*	app 需要主题、安全初始化等	查看 app/build.gradle.kts
feature:* → core:*	feature 可依赖任意 core 模块	查看各 feature 的 build.gradle.kts
core:network → core:database	网络层需读取数据库中的配置 和模型	core/network/build.gradle.kts
core:ui-common → core:common	UI 组件使用公共工具类	core/ui-common/build.gradle.kts
core:security → core:common	安全模块使用公共工具	core/security/build.gradle.kts
禁止	core → feature	核心层不应依赖业务层
禁止	feature:A → feature:B	跨 feature 通过 core:database 共享数 据

核心模块详解

● = 核心模块。修改这些模块可能影响所有 Feature，需谨慎评估。

:core:common — 公共基础 ●

路径: `core/common/src/main/java/com/lianyu/ai/common/`

包名: `com.lianyu.ai.common`

依赖: 仅 AndroidX Core KTX (最小依赖)

被依赖方: 所有其他模块 (13个模块依赖它)

文件清单:

文件	类型	功能说明
<code>ContentFilter.kt</code>	object	内容安全过滤, 正则匹配阻挡违规内容
<code>BanManager.kt</code>	object	用户封禁管理, 检查设备是否被封禁
<code>BatteryOptimizationHelper.kt</code>	object	电池优化白名单、自启动设置引导
<code>FrameRateManager.kt</code>	object	帧率设置持久化与应用
<code>HardwareInfo.kt</code>	object	设备性能分级 (LOW/MEDIUM/HIGH/ULTRA)
<code>ImageUtils.kt</code>	object	图片压缩、格式转换工具
<code>AppForegroundTracker.kt</code>	object	前后台状态追踪 (控制消息推送时机)
<code>update/UpdateModels.kt</code>	数据类	更新检查数据模型 (UpdateInfo、DownloadProgress 等)

关键说明:

- 工具类全部为 `object` 单例, 不持有业务状态
- `HardwareInfo.tier` 用于 UI 组件按设备能力降级 (低端机减少动画)
- `ContentFilter` 的正则模式是安全底线, 修改需 Code Review

`:core:database` — 数据持久化 ●

路径: `core/database/src/main/java/com/lianyu/ai/database/`

包名: `com.lianyu.ai.database`

依赖: Room ORM、Kotlin Serialization、AndroidX Lifecycle ViewModel

被依赖方: 所有 feature 模块 + `core:network`

⚠ 这是项目的数据核心。所有模块通过此模块共享数据模型, 数据库 Schema 变更需要 Migration。

文件清单:

文件	类型	功能说明
<code>AppDatabase.kt</code>	Room Database	数据库入口, 管理所有 DAO 和版本迁移

文件	类型	功能说明
dao/ 数据访问层		
CompanionDao.kt	DAO	AI 伴侣 CRUD
ChatMessageDao.kt	DAO	单聊消息 CRUD
ChatGroupDao.kt	DAO	群聊信息 CRUD
GroupMessageDao.kt	DAO	群聊消息 CRUD
MemoryDao.kt	DAO	记忆条目 CRUD
ApiConfigDao.kt	DAO	API 配置 CRUD
model/ 数据模型 (Entity)		
Companion.kt	Entity	AI 伴侣 (名字、性格、头像等)
ChatMessage.kt	Entity	单聊消息 (内容、时间戳、发送方)
ChatGroup.kt	Entity	群聊信息 (群名、成员列表)
GroupMessage.kt	Entity	群聊消息 (群ID、发送者、内容)
MemoryEntry.kt	Entity	长期记忆 (分类、重要性、访问次数)
ApiConfig.kt	Entity	AI API 配置 (提供商、密钥、模型)
repository/ 仓库层		
CompanionRepository.kt	Repository	伴侣数据操作
ChatRepository.kt	Repository	单聊消息操作
ChatGroupRepository.kt	Repository	群聊信息操作
GroupMessageRepository.kt	Repository	群聊消息操作
MemoryRepository.kt	Repository	记忆操作 (增删查、语义搜索)
UserRepository.kt	Repository	用户信息操作
ApiConfigRepository.kt	Repository	API 配置操作
viewmodel/ 共享 ViewModel		
CompanionListViewModel.kt	ViewModel	伴侣列表 (被 companion + chat + profile 共享)
ChatGroupViewModel.kt	ViewModel	群聊列表 (被 groupchat + profile 共享)

关键说明：

- **为什么 Repository 放在这里：** 多个 Feature 共享同一个 Repository (如 `CompanionRepository` 被

4 个模块使用)

- 数据库版本：当前为 v6，Schema 导出在 `core/database/schemas/`
- 添加新 Entity 的步骤：
 1. 在 `model/` 创建 Entity 类
 2. 在对应 `dao/` 创建 DAO 接口
 3. 在 `AppDatabase.kt` 注册 Entity 和 DAO
 4. 增加数据库版本号并提供 Migration
 5. 在对应 `repository/` 创建 Repository

`:core:network` — 网络通信 ●

路径: `core/network/src/main/java/com/lianyu/ai/network/`
包名: `com.lianyu.ai.network`
依赖: `core:database`、Retrofit、OkHttp、Kotlin Serialization
被依赖方: `feature:chat`、`feature:groupchat`、`feature:notification`、`:app`

文件清单:

文件	类型	功能说明
<code>AIService.kt</code>	服务类	AI 对话主入口，管理多提供商调用和重试逻辑

关键说明:

- `AIService` 从 `core:database` 读取 `ApiConfig` 确定当前使用的 AI 提供商
- 网络层不持有 UI 状态，返回结果通过 Flow 或 suspend 函数
- 请求超时 15s 连接 + 60s 读取，超时自动重试

`:core:security` — 安全加密 ●

路径: `core/security/src/main/`
包名: `com.lianyu.ai.security`
依赖: 仅 AndroidX Core KTX
被依赖方: `:app` (Application 初始化)、`feature:update`

⚠ 此模块包含 JNI/NDK 编译。修改 C++ 代码或编译配置需格外谨慎。

文件清单:

文件	类型	功能说明
<code>NativeBridge.kt</code>	object	JNI 桥接，加载 <code>.so</code> 并提供 native 方法
<code>SecurityGuard.kt</code>	object	运行时安全检查（防调试、防篡改）

文件	类型	功能说明
<code>native-bridge.cpp</code>	C++	Native 层实现（签名验证、请求注入）
<code>Android.mk</code>	NDK	Android.mk 编译配置
<code>Application.mk</code>	NDK	Application.mk 编译配置
<code>version-script.map</code>	Linker	符号导出控制

NDK 编译配置：

```
// core/security/build.gradle.kts
ndkVersion = "30.0.14904198" // 必须与 app 模块一致
defaultConfig {
    ndk {
        abiFilters += listOf("arm64-v8a", "armeabi-v7a", "x86_64", "x86")
    }
}
externalNativeBuild {
    ndkBuild {
        path("src/main/cpp/Android.mk")
    }
}
```

Native 方法列表：

方法	功能
<code>verifySignature(Context)</code>	验证 APK 签名，防二次打包
<code>injectAuthHeader(Any)</code>	向 OkHttp 请求注入认证头
<code>getRepoOwner()</code>	获取 GitHub 仓库所有者
<code>getRepoName()</code>	获取 GitHub 仓库名
<code>getGitHubApiUrl()</code>	获取 GitHub API 地址
<code>isSafe()</code>	综合安全检测
<code>isMitmDetected()</code>	中间人攻击检测
<code>verifyRequestIntegrity(String)</code>	请求完整性校验

关键说明：

- `System.loadLibrary("lianyu_security")` 在 `NativeBridge.init` 中执行，如果 `.so` 未编译会导致应用闪退
- 签名校验仅在 release 包生效（debug 包跳过）
- 更改 NDK 版本需与 `app/build.gradle.kts` 中的 `ndkVersion` 保持一致

`:core:ui-common` — 通用 UI

路径: `core/ui-common/src/main/java/com/lianyu/ai/uicommon/`

包名: `com.lianyu.ai.uicommon`

依赖: `core:common`、Compose BOM、Material 3、Coil

被依赖方: 所有 feature 模块 + `:app`

 修改主题系统会影响整个应用的视觉呈现。

文件清单:

文件	类型	功能说明
<code>theme/</code> 主题系统		
<code>Theme.kt</code>	Composable	主题入口 <code>LianYuTheme</code> (亮/暗/跟随系统)
<code>Color.kt</code>	常量	微信风格色板 + 旧版色值 (保持兼容)
<code>Type.kt</code>	常量	字体排版定义 <code>AppTypography</code>
<code>component/</code> 共享组件		
<code>LiquidGlass.kt</code>	Composable	液态玻璃特效 (<code>frostedGlassNav</code> 、 <code>liquidGlass</code> 、 <code>ellipseLiquidGlass</code>)
<code>AnimatedBackground.kt</code>	Composable	动态动画背景
<code>SpringAnimations.kt</code>	Modifier 扩展	弹簧动画 (<code>bounceIn</code> 、 <code>pulseScale</code> 、 <code>floatAnimation</code> 、 <code>glowPulse</code>)
<code>ShimmerEffect.kt</code>	Composable	Shimmer 骨架屏效果
<code>ChatBackground.kt</code>	Composable	聊天背景选择与管理
<code>ChatBackgroundCache.kt</code>	缓存	聊天背景图片缓存
<code>WeChatChatInputBar.kt</code>	Composable	微信风格输入栏组件
<code>UpdateDialog.kt</code>	Composable	应用更新弹窗

关键说明:

- `WeChatLightBackground` / `WeChatDarkBackground` 是全局背景色
- `WeChatGreen` 为用户气泡色, `AiBubbleLight` / `AiBubbleDark` 为 AI 气泡色
- `HardwareInfo.tier` 用于控制动画强度 (低端机关闭液态玻璃等重特效)
- 新增通用组件放在 `component/`, 仅某 Feature 使用的放在该 Feature 模块内

功能模块详解

:feature:chat — 一对一聊天

路径: `feature/chat/src/main/java/com/lianyu/ai/feature/chat/`
包名: `com.lianyu.ai.feature.chat`
依赖: `core:common`、`core:database`、`core:network`、`core:ui-common`、`feature:profile` (读取用户头像)

文件清单:

文件	功能
<code>ui/screen/ChatScreen.kt</code>	聊天主界面 (消息列表、输入栏、导航栏)
<code>ui/viewmodel/ChatViewModel.kt</code>	聊天逻辑 (发送消息、接收 AI 回复、消息持久化)
<code>ui/viewmodel/ChatViewModelFactory.kt</code>	ViewModel Factory (传递 companionId)

数据流:

```
ChatScreen
  ↓ user input
ChatViewModel.sendMessage()
  ↓ save to DB
ChatRepository ← AppDatabase
  ↓ get history
AiService.sendMessage() ← core:network
  ↓ AI response
ChatRepository.sendMessage() → DB
  ↓ Flow emit
ChatScreen (UI update)
```

关键说明:

- `ChatViewModel` 需要 `companionId` 构造参数, 使用 `ChatViewModelFactory`
- 消息列表限制显示最近 30 条, 通过 DAO 的 Flow 实时更新
- 性能优化: 导航动画完成后 (200ms delay) 再渲染消息列表

:feature:companion — 伴侣管理

路径: `feature/companion/src/main/java/com/lianyu/ai/feature/companion/`
包名: `com.lianyu.ai.feature.companion`
依赖: `core:common`、`core:database`、`core:ui-common`

文件清单:

文件	功能
<code>ui/screen/CompanionListScreen.kt</code>	伴侣列表页（动画入场、滑动手势）
<code>ui/screen/ContactsScreen.kt</code>	通讯录页（伴侣 + 群聊混合展示）
<code>ui/screen/CreateCompanionScreen.kt</code>	创建/编辑伴侣（名字、性格、头像选择）
<code>ui/viewmodel/CreateCompanionViewModel.kt</code>	创建伴侣逻辑（图片处理、数据持久化）

关键说明：

- 创建/编辑复用同一个 Screen，通过 `companionId` 参数区分
- 伴侣头像通过 `ImageUtils` 压缩后保存

`:feature:groupchat` — 群聊

路径：`feature/groupchat/src/main/java/com/lianyu/ai/feature/groupchat/`

包名：`com.lianyu.ai.feature.groupchat`

依赖：`core:common`、`core:database`、`core:network`、`core:ui-common`、`feature:profile`

文件清单：

文件	功能
<code>ui/GroupChatScreen.kt</code>	群聊界面（多 AI 同时回复）
<code>ui/CreateGroupScreen.kt</code>	创建群聊（选择成员、设置群名）
<code>GroupChatViewModel.kt</code>	群聊逻辑（群发消息、收集各 AI 回复）
<code>GroupChatViewModelFactory.kt</code>	ViewModel Factory（传递 groupId）

关键说明：

- 群聊中每个 AI 伴侣独立回复，消息并发生成
- 消息通过 `GroupMessage` 存储，通过 `companionId` 区分发送者

`:feature:memory` — 记忆系统

路径：`feature/memory/src/main/java/com/lianyu/ai/feature/memory/`

包名：`com.lianyu.ai.feature.memory`

依赖：`core:common`、`core:database`、`core:ui-common`

文件清单：

文件	功能
<code>MemoryScreen.kt</code>	记忆展示页（按伴侣/分类筛选、删除管理）

文件	功能
MemoryViewModel.kt	记忆逻辑（获取记忆、删除、按类别查询）

关键说明：

- 记忆分为 `FACT`（事实）、`PREFERENCE`（偏好）、`RELATIONSHIP`（关系）等类别
- 支持语义搜索（通过 `MemoryRepository.searchMemories()`）

`:feature:notification` — 通知与后台

路径：`feature/notification/src/main/java/com/lianyu/ai/feature/notification/`

包名：`com.lianyu.ai.feature.notification`

依赖：`core:common`、`core:database`、`core:network`

⚠ 此模块涉及后台服务和 AndroidManifest 声明，修改 Service/Receiver 后需同步更新 `app/src/main/AndroidManifest.xml`。

文件清单：

文件	功能
NotificationHelper.kt	通知创建与展示
CompanionKeepAliveService.kt	前台服务保活（防止进程被杀）
CompanionMessageWorker.kt	WorkManager 定时任务（AI 主动发消息）
AiReplyWorker.kt	AI 回复处理 Worker
BootReceiver.kt	开机自启动广播

AndroidManifest 声明（在 `app/src/main/AndroidManifest.xml`）：

```
<!-- 必须使用全限定类名！ -->
<service android:name="com.lianyu.ai.feature.notification.CompanionKeepAliveService"
    android:foregroundServiceType="dataSync" />
<receiver android:name="com.lianyu.ai.feature.notification.BootReceiver" />
```

关键说明：

- 消息间隔根据亲密度动态调整：亲密度越高，消息越频繁
- `ExistingPeriodicWorkPolicy.UPDATE`（非 `REPLACE`，已弃用）
- 前台通知必须启用（Android 14+ 强制要求 `foregroundServiceType`）

`:feature:profile` — 个人中心与首页

路径: `feature/profile/src/main/java/com/lianyu/ai/feature/profile/`
包名: `com.lianyu.ai.feature.profile`
依赖: `core:common`、`core:database`、`core:ui-common`

文件清单:

文件	功能
<code>HomeScreen.kt</code>	首页（聊天预览、伴侣/群聊快速入口）
<code>ProfileScreen.kt</code>	个人中心（用户头像、设置入口）
<code>AboutScreen.kt</code>	关于页面
<code>AgreementScreen.kt</code>	初次启动的用户协议
<code>AgreementViewScreen.kt</code>	协议查看页
<code>BanScreen.kt</code>	封禁拦截页
<code>HomeViewModel.kt</code>	首页数据逻辑（最近消息预览）
<code>ProfileViewModel.kt</code>	个人中心数据（头像、昵称管理）

关键说明:

- `AgreementScreen` 是应用启动的第一个页面（未同意协议时阻塞进入）
- `BanScreen` 是封禁拦截（在协议之后、主页之前）

`:feature:settings` — 设置中心

路径: `feature/settings/src/main/java/com/lianyu/ai/feature/settings/`
包名: `com.lianyu.ai.feature.settings`
依赖: `core:common`、`core:database`、`core:ui-common`、`feature:update`

文件清单:

文件	功能
<code>ui/screen/SettingsScreen.kt</code>	设置主页（选项列表）
<code>ui/screen/ThemeScreen.kt</code>	主题切换（亮/暗/跟随系统）
<code>ui/screen/LanguageScreen.kt</code>	多语言切换（5种语言）
<code>ui/screen/FrameRateScreen.kt</code>	帧率设置（60/90/120/自动）
<code>ui/screen/CheckUpdateScreen.kt</code>	检查更新页面
<code>ui/viewmodel/SettingsViewModel.kt</code>	设置逻辑

文件	功能
ui/viewmodel/ThemeViewModel.kt	主题逻辑
ui/viewmodel/LanguageViewModel.kt	语言逻辑

关键说明：

- 语言切换需要 `activity.recreate()` 重新加载资源
- 帧率设置通过 `FrameRateManager` 全局生效

:app 入口模块

路径: `app/src/main/java/com/lianyu/ai/`

包名: `com.lianyu.ai`

依赖: 所有 core 模块 + 所有 feature 模块

仅包含 2 个 Kotlin 文件：

文件	功能
LianYuApplication.kt	Application：安全初始化、语言设置、背景预加载
MainActivity.kt	Activity：全屏配置、NavHost 路由表、更新弹窗逻辑

启动链路：

```
LianYuApplication.onCreate()
├── SecurityGuard.init()           ← 安全校验
├── NativeBridge.verifySignature() ← 签名验证
├── AiService.initialize()         ← 网络预热
├── applyStoredLanguage()          ← 加载已保存语言
└── preloadBackground()           ← 异步加载聊天背景

MainActivity.onCreate()
├── setContent { LianYuTheme {
│   ├── AgreementScreen           ← 首次启动：用户协议
│   ├── BanScreen                 ← 封禁拦截
│   └── MainScreen
│       └── NavHost               ← 全局路由
│           ├── "home"            → HomeScreen
│           ├── "contacts"        → ContactsScreen
│           ├── "profile"         → ProfileScreen
│           ├── "chat/{id}"       → ChatScreen
│           ├── "group_chat/{id}" → GroupChatScreen
│           ├── "create"          → CreateCompanionScreen
│           ├── "create_group"    → CreateGroupScreen
│           ├── "settings"        → SettingsScreen
│           └── "memory"          → MemoryScreen
│       }
│   }
└── }
```

```
├── "theme"      → ThemeScreen
├── "language"   → LanguageScreen
├── "about"      → AboutScreen
├── ...
}}
├── CompanionKeepAliveService.start() ← 启动后台保活
├── AppUpdateManager.checkForUpdates() ← 检查更新
```

导航路由表（完整）：

路由	页面	模块	参数
home	首页	feature:profile	-
contacts	通讯录	feature:companion	-
profile	个人中心	feature:profile	-
chat/{companionId}	聊天	feature:chat	Long
create	创建伴侣	feature:companion	-
edit/{companionId}	编辑伴侣	feature:companion	Long
group_chat/{groupId}	群聊	feature:groupchat	Long
create_group	创建群聊	feature:groupchat	-
settings	设置	feature:settings	-
memory	记忆	feature:memory	-
theme	主题	feature:settings	-
language	语言	feature:settings	-
check_update	检查更新	feature:settings	-
frame_rate	帧率	feature:settings	-
about	关于	feature:profile	-
agreement_view	协议查看	feature:profile	-

模块创建规范

Step 1: 创建模块目录

```
feature/
├── newfeature/
```

```

├─ build.gradle.kts
├─ src/main/
│   └─ AndroidManifest.xml          # module namespace
│       └─ java/com/lianyu/ai/feature/newfeature/
│           ├── ui/
│           │   ├── screen/          # Compose Screen
│           │   ├── component/       # 模块内共享组件
│           │   └─ viewmodel/        # ViewModel
│           ├── data/
│           │   └─ repository/       # Repository (如需)
│           └─ di/                   # Hilt Module (如使用 DI)

```

Step 2: 编写 `build.gradle.kts`

```

plugins {
    alias(libs.plugins.android.library)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.serialization)
}

android {
    namespace = "com.lianyu.ai.feature.newfeature"
    compileSdk = 35

    defaultConfig { minSdk = 26 }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    buildFeatures { compose = true }
    composeOptions { kotlinCompilerExtensionVersion = "1.5.4" }
    kotlin { jvmToolchain(17) }
}

dependencies {
    // 根据需求选择 core 模块
    implementation(project(":core:common"))           // 必须
    implementation(project(":core:database"))          // 数据操作时需要
    implementation(project(":core:ui-common"))         // UI 时需要
    implementation(project(":core:network"))           // 网络调用时需要

    // Compose
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.lifecycle.viewmodel.compose)
    implementation(libs.androidx.navigation.compose)
    implementation(libs.androidx.compose.material.icons.extended)
    implementation(libs.coil.compose)
}

```

```
implementation(libs.kotlin.serialization.json)
}
```

Step 3: 注册模块到项目

在 `settings.gradle.kts` 的 `include` 块中添加:

```
include(":feature:newfeature")
```

Step 4: 在 `:app` 中依赖

```
// app/build.gradle.kts
dependencies {
    implementation(project(":feature:newfeature"))
    // ... 其他 feature 模块
}
```

Step 5: 添加导航路由

在 `:app` 的 `MainActivity.kt` 中注册新页面路由:

```
composable("newfeature") {
    NewFeatureScreen(
        onNavigateBack = { navController.popBackStack() }
    )
}
```

跨模块通信规范

禁止直接依赖

- Feature 模块之间 **禁止直接依赖**
- 例如 `:feature:chat` 不能 `implementation(project(":feature:companion"))`

通过数据库共享数据

跨 Feature 的数据共享通过 `core-database` 实现:

```
// feature:chat 中需要获取 Companion 信息
class ChatRepository(
    private val companionDao: CompanionDao, // 来自 core-database
    private val chatMessageDao: ChatMessageDao
) {
    suspend fun getCompanion(companionId: Long): Companion? {
```

```
        return companionDao.getById(companionId)
    }
}
```

通过回调/事件传递

UI 层通过导航回调传递简单数据：

```
// MainActivity.kt 中
composable("chat/{companionId}") { backStackEntry ->
    val companionId = backStackEntry.arguments?.getLong("companionId") ?: 0L
    ChatScreen(
        companionId = companionId,
        onNavigateBack = { navController.popBackStack() }
    )
}
```

开发流程与最佳实践

1. 开发新功能时的模块选择

场景	建议
新增一个设置项	放到现有 <code>feature:settings</code> 模块
新增独立功能（如语音通话）	新建 <code>feature:voicecall</code> 模块
新增通用 UI 组件（如图片选择器）	放到 <code>core:ui-common</code> 或新建 <code>core:media</code>
新增数据模型被多模块使用	放到 <code>core:database</code>

2. 数据库 Schema 变更流程

1. 修改 Entity 类（添加/删除字段）
2. 创建 Migration 类（如 Migration(6, 7)）
3. 在 AppDatabase.kt 中注册 Migration
4. 增加数据库版本号
5. 导出 Schema（./gradlew room:exportSchema）
6. 全量编译验证

3. 添加新导航页面的标准流程

1. 在目标 Feature 模块创建 Screen Composable
2. 在 :app 的 MainActivity.kt 添加 composable 路由

3. 从其他页面通过 `navController.navigate("route")` 跳转
4. 使用 `navController.popBackStack()` 返回

4. 调试依赖关系

```
# 查看 app 模块的所有依赖
./gradlew app:dependencies --configuration implementation

# 查看某个 feature 的依赖树
./gradlew feature:chat:dependencies --configuration implementation

# 检查循环依赖
./gradlew build --dry-run
```

5. 性能优化 checklist

- [] 低端机 (`HardwareInfo.tier == LOW`) 关闭液态玻璃等重特效
- [] 聊天消息列表使用 LazyColumn + key
- [] 图片加载使用 Coil 的缓存策略
- [] 数据库查询使用 Flow 避免手动刷新
- [] WorkManager 任务使用 `ExistingPeriodicWorkPolicy.UPDATE`

常见问题

Q: 为什么 Entity 要放在 core-database 而不是各自的 Feature 模块?

A: 因为多个 Feature 需要共享 Entity。例如 `Companion` 被 `feature:companion`、`feature:chat`、`feature:groupchat`、`feature:memory` 等多个模块使用。如果放在某个 Feature 中, 其他 Feature 就需要依赖它, 违反 Feature 间不直接依赖的原则。

Q: Repository 为什么放在 core-database 而不是 Feature 模块?

A: 因为 Repository 被多个 Feature 共享。例如 `CompanionRepository` 被 4 个模块使用。如果放在某个 Feature 中, 其他 Feature 就需要依赖它, 违反 Feature 间不直接依赖的原则。

Q: 新增一个功能, 需要创建新模块吗?

A: 视复杂度而定:

- 简单功能 (如一个设置项的开关) → 放到现有 `feature:settings` 模块
- 独立功能 (如语音通话) → 新建 `feature:voicecall` 模块
- 通用能力 (如图片选择器) → 放到 `core:ui-common` 或新建 `core:media`

Q: 如何调试模块间的依赖问题?

A: 使用 Gradle 任务检查依赖关系:

```
./gradlew app:dependencies --configuration implementation | grep feature
```

Q: 修改 core 模块后需要全量编译吗?

A: 是的。core 模块被所有 feature 模块依赖, 修改后所有依赖它的模块都需要重新编译。建议:

- 小改动: 直接 `./gradlew assembleDebug`
- 大改动: 先 `./gradlew clean` 再编译

Q: 为什么 app 闪退, log 显示 `UnsatisfiedLinkError` ?

A: 这是 `core:security` 的 `.so` 文件未编译成功。检查:

1. NDK 是否正确安装 (`D:\Android\Sdk\ndk\30.0.14904198`)
2. `core/security/build.gradle.kts` 中 `ndkVersion` 是否与 app 一致
3. `Android.mk` 路径是否正确

Q: 新增模块后编译报错 "Project with path ':feature:xxx' could not be found"?

A: 忘记在 `settings.gradle.kts` 中 `include` 新模块。每个新模块必须在这里注册。

附录: 完整文件清单

core 模块

```
core/common/src/main/java/com/lianyu/ai/common/
├── BanManager.kt
├── BatteryOptimizationHelper.kt
├── ContentFilter.kt
├── FrameRateManager.kt
├── HardwareInfo.kt
├── ImageUtils.kt
├── AppForegroundTracker.kt
├── update/
│   └── UpdateModels.kt
└──

core/database/src/main/java/com/lianyu/ai/database/
├── AppDatabase.kt
├── dao/
│   ├── CompanionDao.kt
│   ├── ChatMessageDao.kt
│   ├── ChatGroupDao.kt
│   └── GroupMessageDao.kt
└──
```

```
|   |   | MemoryDao.kt
|   |   | ApiConfigDao.kt
|   | model/
|   |   | Companion.kt
|   |   | ChatMessage.kt
|   |   | ChatGroup.kt
|   |   | GroupMessage.kt
|   |   | MemoryEntry.kt
|   |   | ApiConfig.kt
|   | repository/
|   |   | CompanionRepository.kt
|   |   | ChatRepository.kt
|   |   | ChatGroupRepository.kt
|   |   | GroupMessageRepository.kt
|   |   | MemoryRepository.kt
|   |   | UserRepository.kt
|   |   | ApiConfigRepository.kt
|   | viewmodel/
|   |   | CompanionListViewModel.kt
|   |   | ChatGroupViewModel.kt
|
| core/network/src/main/java/com/lianyu/ai/network/
|   | AiService.kt
|
| core/security/src/main/
|   | java/com/lianyu/ai/security/
|   |   | NativeBridge.kt
|   |   | SecurityGuard.kt
|   | cpp/
|   |   | native-bridge.cpp
|   |   | Android.mk
|   |   | Application.mk
|   |   | version-script.map
|
| core/ui-common/src/main/java/com/lianyu/ai/uicommon/
|   | theme/
|   |   | Theme.kt
|   |   | Color.kt
|   |   | Type.kt
|   | component/
|   |   | LiquidGlass.kt
|   |   | AnimatedBackground.kt
|   |   | SpringAnimations.kt
|   |   | ShimmerEffect.kt
|   |   | ChatBackground.kt
|   |   | ChatBackgroundCache.kt
|   |   | WeChatChatInputBar.kt
|   |   | UpdateDialog.kt
```

feature 模块

```
| feature/chat/src/main/java/com/lianyu/ai/feature/chat/
|   | ui/screen/ChatScreen.kt
```

```
├─ ui/viewmodel/ChatViewModel.kt
└─ ui/viewmodel/ChatViewModelFactory.kt

feature/companion/src/main/java/com/lianyu/ai/feature/companion/
├─ ui/screen/CompanionListScreen.kt
├─ ui/screen/ContactsScreen.kt
├─ ui/screen/CreateCompanionScreen.kt
└─ ui/viewmodel/CreateCompanionViewModel.kt

feature/groupchat/src/main/java/com/lianyu/ai/feature/groupchat/
├─ ui/GroupChatScreen.kt
├─ ui/CreateGroupScreen.kt
├─ GroupChatViewModel.kt
└─ GroupChatViewModelFactory.kt

feature/memory/src/main/java/com/lianyu/ai/feature/memory/
├─ MemoryScreen.kt
└─ MemoryViewModel.kt

feature/notification/src/main/java/com/lianyu/ai/feature/notification/
├─ NotificationHelper.kt
├─ CompanionKeepAliveService.kt
├─ CompanionMessageWorker.kt
├─ AiReplyWorker.kt
└─ BootReceiver.kt

feature/profile/src/main/java/com/lianyu/ai/feature/profile/
├─ HomeScreen.kt
├─ ProfileScreen.kt
├─ AboutScreen.kt
├─ AgreementScreen.kt
├─ AgreementViewScreen.kt
├─ BanScreen.kt
├─ HomeViewModel.kt
└─ ProfileViewModel.kt

feature/settings/src/main/java/com/lianyu/ai/feature/settings/
├─ ui/screen/SettingsScreen.kt
├─ ui/screen/ThemeScreen.kt
├─ ui/screen/LanguageScreen.kt
├─ ui/screen/FrameRateScreen.kt
├─ ui/screen/CheckUpdateScreen.kt
├─ ui/viewmodel/SettingsViewModel.kt
├─ ui/viewmodel/ThemeViewModel.kt
└─ ui/viewmodel/LanguageViewModel.kt
```

app 模块

```
app/src/main/java/com/lianyu/ai/
├─ LianYuApplication.kt
└─ MainActivity.kt
```

相关文档

- [README.md](#) — 项目简介
- [CONTRIBUTING.md](#) — 贡献者指南（待创建）

参考资源

- [Android 官方模块化指南](#)
- [Now in Android 模块化架构](#)